

VNU-HCM University of Information Technology

ASSIGNMENT 5

By Chu Cuong - 24520236

Email: cuongchu200@gmail.com
5-15-2025

Mục Lục

Lời mở đầu.....	2
1. H007 – Alpha Problem	3
2. H006 – Word Merging.....	5
3. H004 – Wood Cutting.....	7
4. H003 – Isosceles Triangle	9
5. H002 – Difference in Height	11
6. H001 – Tom’s Currency.....	13
7. Hashing: VQ44_FLOWERS	15
8. Hashing: KiemKe	17

My Github: [LINK](#)

Lời mở đầu

Bài tập tuần 5 chỉ có phần Basic với 8 bài tập được giao nhằm mục đích ôn lại kiến thức đã học. Sau đây là phần trình bày và cách giải các bài tập của mình:

[IT003.P21.CTTN.2] Assignment 5 (Basic) 

8 problems with a total score of 800

#	PROBLEM	SCORE
1	[H007] - Alpha Problem	100
2	[H006] - Word Merging	100
3	[H004] - Wood Cutting	100
4	[H003] - Isosceles Triangle	100
5	[H002] - Difference in Height	100
6	[H001] - Tom's Currency	100
7	Hashing: VQ44_FLOWERS	100
8	Hashing: KiemKe	100

1. H007 – Alpha Problem

➤ Đề bài:

University of Information Technology, VNU-HCM
UIT Algo Bootcamp

Rookie Training
Spring 2023

Alpha's Problem

Một ngày đẹp trời nọ, đó là khi lớp Alpha đang có tiết Tin học, nội dung ngày hôm ấy là về cách chuyển đổi số giữa các hệ cơ số với nhau. Do quá mải mê nhắn tin với bạn gái nên Alpha không tập trung nội dung mà giáo viên đang giảng dạy. Xui cho Alpha rằng hành động đó đã bị giáo viên phát hiện, Alpha bị kêu lên bảng làm bài, nếu Alpha không làm được thì sẽ bị ghi tên vào sổ đầu bài.

Giáo viên cho Alpha một số tự nhiên a được biểu diễn trong hệ thập phân và yêu cầu chuyển số đó sang hệ x phân. Do Alpha không giỏi về mặt toán và tin học nên một lần nữa bạn hãy ra tay tính giúp Alpha nhé.

Input

Gồm một dòng duy nhất chứa hai số nguyên a ($0 \leq a \leq 10^{18}$) và x cách nhau bởi khoảng trắng.

Output

Gồm một dòng duy nhất là biểu diễn của số a trong hệ x phân.

Example

input	output
12345 6	133053
input	output
987654321 2	111010110111100110100010110001

Subtask

- 10% số bộ kiểm tra có $x = 10$.
- 30% số bộ kiểm tra có $x = 2$.
- 60% số bộ kiểm tra có $3 \leq x \leq 9$.

➤ Ý tưởng:

Với mục tiêu chuyển số thập phân được nhập vào thành 1 giá trị ở hệ mới quy định từ 1 đến 10 thì ta có thể nghĩ tới phương pháp chia lấy dư để hoàn thành bài này.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    ull a, x;
    cin >> a >> x;
    if(x == 10){
        cout << a;
        return 0;
    }

    vector<char> v;
    while(a != 0){
        short temp = a % x;
        v.push_back('0' + temp);
        a = a / x;
    }

    if(v.empty()) cout << 0;
    else{
        for(int i = v.size() - 1; i >= 0; i--)
            cout << v[i];
    }
    return 0;
}
```

2. H006 – Word Merging

➤ Đề bài:

University of Information Technology, VNU-HCM
UIT Algo Bootcamp

Rookie Training
Spring 2023

Nối chữ

Cho xâu s có độ dài $|s|$, có dạng $s_1s_2...s_n$, chỉ bao gồm các chữ cái tiếng Anh viết thường.

Bạn được phép thực hiện thao tác sau đây với số lần **không giới hạn**: chọn hai chữ cái giống nhau liên tiếp trong s và xoá chúng. Sau thao tác này, các chữ cái còn lại sẽ được nối với nhau.

Nói cách khác, gọi m là độ dài của xâu s hiện tại, bạn có thể chọn vị trí i với $1 \leq i \leq m - 1$ sao cho $s_i = s_{i+1}$ và xoá chúng. Sau khi xoá, xâu s sẽ có độ dài $m - 2$ và có dạng $s_1s_2s_3...s_{i-1}s_{i+1}s_{i+2}...s_m$.

Ví dụ, xét xâu $s = aaabccddd$

Ở lần đầu tiên, ta chọn vị trí 5:

$aaab\text{c}cddd \rightarrow aaabddd$

Ở lần thứ hai, ta chọn vị trí 2:

$a\text{a}abddd \rightarrow abddd$

Ở lần thứ ba, ta chọn vị trí 3:

$ab\text{d}dd \rightarrow abd$

Yêu cầu: Hãy tính độ dài xâu s ngắn nhất có thể sau khi thực hiện các thao tác như trên.

Input

- Gồm một dòng duy nhất chứa xâu s .

Output

- Gồm một số nguyên không âm duy nhất là độ dài xâu s ngắn nhất có thể sau khi thực hiện các thao tác như trên.

Example

input	output
aaabccddd	3
input	output
baab	0

Subtask

- 50% số test có $1 \leq |s| \leq 10^3$.

- 50% số test khác có $1 \leq |s| \leq 2 \times 10^5$.

➤ Ý tưởng:

Để xóa hai chữ cái giống nhau liên tiếp thì ta sẽ duyệt qua xâu kí tự và đánh dấu phần tử cuối cùng rồi so sánh với phần tử tiếp theo.

Và khi mà phần tử tiếp theo trùng với giá trị được đánh dấu thì ta xóa phần tử trên cùng và đánh dấu lại phần tử cuối cùng để tìm phần tử trùng lặp tiếp theo.

Với việc so và duyệt chuỗi như vậy thì chuỗi cuối cùng ta nhận lại là kết quả.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    string s;
    cin >> s;
    stack<char> st;
    for (char c : s){
        if(!st.empty() && st.top() == c)
            st.pop();
        else st.push(c);
    }
    cout << st.size();
    return 0;
}
```

3. H004 – Wood Cutting

➤ Đề bài:

University of Information Technology, VNU-HCM
UIT Algo Bootcamp

Rookie Training
Spring 2023

Cắt gỗ

Ông Hai có một khúc gỗ có chiều dài S . Ông muốn cắt khúc gỗ thành n phần với chiều dài lần lượt là $a_1, a_2, \dots, a_n$.

Chi phí để chia khúc gỗ thành hai phần là chiều dài của khúc gỗ đó.

Ví dụ: Ông Hai muốn chia khúc gỗ có chiều dài 16 thành ba phần 1, 9, 6.

Bước một: cắt khúc gỗ có kích thước 16 thành hai phần 9 và 7 với chi phí là 16.

Bước hai: cắt khúc gỗ có kích thước 7 thành hai phần 6 và 1 với chi phí là 7.

Tổng chi phí cho cách cắt này là $16 + 7 = 23$, đồng thời đây là cách cắt tối ưu cho chi phí thấp nhất.

Yêu cầu: Hãy giúp Ông Hai tính chi phí cắt gỗ thấp nhất.

Input

- Dòng đầu gồm hai số nguyên dương n và S ($1 \leq n \leq 2 \times 10^5$).

- Dòng sau gồm n số nguyên dương $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

Input đảm bảo $\sum_{i=1}^n a_i = S$

Output

- Gồm một số nguyên dương duy nhất là chi phí cắt gỗ thấp nhất.

Example

input	output
3 16 1 6 9	23
input	output
5 24 8 1 3 5 7	52

➤ Ý tưởng:

Yêu cầu bài này giống bài toán hợp nhất đoạn ([Optimal File Merge Patterns](#))

Để tìm chi phí tính toán tối thiểu, chúng ta cần phải sắp xếp mảng, thêm chi phí tính toán tối thiểu có thể và xóa chúng khỏi mảng. Chúng ta làm bài này tối ưu bằng cách sử dụng cấu trúc dữ liệu min-heap (priority-queue).

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define fi first
#define se second
#define ull unsigned long long

ull minCutCost(vector<int>& v) {
    priority_queue<ull, vector<ull>, greater<ull>> pq(v.begin(), v.end());

    ull totalCost = 0;
    while (pq.size() > 1) {
        ull fi = pq.top();
        pq.pop();
        ull se = pq.top();
        pq.pop();

        ull cost = fi + se;
        totalCost += cost;
        pq.push(cost);
    }

    return totalCost;
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    ull S;
    cin >> n >> S;
    vector<int> v(n);
    for(int i = 0; i < n; i++){
        cin >> v[i];
    }
    cout << minCutCost(v);
    return 0;
}
```

4. H003 – Isosceles Triangle

➤ Đề bài:

University of Information Technology, VNU-HCM
UIT Algo Bootcamp

Rookie Training
Spring 2023

Tam giác cân

Tam giác cân là tam giác có ít nhất 2 cạnh có độ dài bằng nhau.

Cho dãy gồm N số nguyên dương: $a_1, a_2, \dots, a_N$. Hãy tính số bộ 3 chỉ số (i, j, k) , với $1 \leq i < j < k \leq N$ sao cho 3 số a_i, a_j, a_k là độ dài 3 cạnh của một tam giác cân.

Input

- Dòng đầu ghi số nguyên N ($3 \leq N \leq 5 \times 10^5$).
- Dòng tiếp theo ghi N số nguyên dương là các phần tử của dãy, giá trị không vượt quá 10^9 .

Output

- Gồm một dòng chứa số lượng bộ ba chỉ số phân biệt (i, j, k) , với $1 \leq i < j < k \leq N$ sao cho 3 số a_i, a_j, a_k là độ dài 3 cạnh của một tam giác cân.

Example

input	output
8 5 3 2 9 5 4 9 5	22

Subtask

- 30% số điểm có ($1 \leq N \leq 100$).
- 30% số điểm có ($1 \leq N \leq 10^5$), ($1 \leq a_i \leq 10^5$).
- 40% số điểm còn lại có giới hạn như đề bài.

➤ Ý tưởng:

Ta sẽ đếm số lượng cạnh x có độ dài bằng nhau và cho vào map, tiếp theo ta sẽ đếm số lượng cạnh có độ dài nhỏ hơn x , với mọi x . Làm vậy là để khi đếm số tam giác cân với 2 cạnh bằng nhau là x , ta sẽ xét 2 trường hợp:

- TH1: 3 cạnh có cùng độ dài x .
- TH2: 2 cạnh có độ dài x , cạnh thứ 3 là a thỏa mãn điều kiện $2x = x + x > a$

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int N = 1e6 + 9;
```

```

ll nC2(int n) {
    return (n < 2) ? 0 : (ll)n * (n - 1) / 2;
}

ll nC3(int n) {
    return (n < 3) ? 0 : (ll)n * (n - 1) * (n - 2) / 6;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    cin >> t;
    map<int, pair<int, int>> freq;
    int max_val = 0;
    for (int i = 0; i < t; ++i) {
        int x;
        cin >> x;
        freq[x].first++;
    }

    for (auto x = freq.begin(); x != freq.end(); x++) {
        if (x == freq.begin()) {
            x->second.second = x->second.first;
            continue;
        }
        auto y = x;
        y--;
        x->second.second = y->second.second + x->second.first;
    }

    ll sum = 0;
    for (auto &x : freq) {
        auto it = freq.lower_bound(x.first * 2);
        it--;
        sum += nC2(x.second.first) * (it->second.second - x.second.first) +
nC3(x.second.first);
    }

    cout << sum;
    return 0;
}

```

5. H002 – Difference in Height

➤ Đề bài:

University of Information Technology, VNU-HCM
UIT Algo Bootcamp

Rookie Training
Spring 2023

Chênh lệch

Thành phố Alpha có n toà nhà cao tầng, được đánh số từ 1 đến n . Toà nhà cao tầng thứ i có độ cao là a_i . Độ chênh lệch độ cao của 2 toà nhà i, j là hiệu số độ cao giữa hai toà nhà đó, hay nói cách khác là $|a_i - a_j|$.

Để phục vụ cho công tác quản lý thành phố, chính quyền cần tính tổng độ chênh lệch giữa từng đôi một toà nhà. Hay nói cách khác là $\sum_{1 \leq i < j \leq n} |a_i - a_j|$.

Hãy tính kết quả mà chính quyền cần.

Input

Dòng đầu tiên chứa số nguyên n là số toà nhà cao tầng trong thành phố ($1 \leq n \leq 2 \times 10^5$).

Dòng thứ hai chứa n số nguyên bao gồm các số $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

Output

Gồm 1 số duy nhất là kết quả cần tìm.

Example

input	output
3	194
100 3 7	

➤ Ý tưởng:

Với mỗi cặp số (a, b) thì chỉ đếm có 1 lần nên khi ta xét theo từng giá trị thì nó sẽ có mặt trong $n - 1$ cặp còn lại. Và với giá trị thứ i trong dãy được sắp xếp của n phần tử thì nó sẽ có hai trường hợp:

- TH1: Số bị trừ trong cặp với $i - 1$ phần tử bé hơn
- TH2: Số trừ trong cặp với $n - i$ phần tử lớn hơn

⇒ Với mỗi i , nó sẽ thêm vào tổng là: $[i - 1 - (n - i)]a[i] = (2i - n - 1)a[i]$

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;
#define ull unsigned long long

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    vector<int> v(n);
    for(int i = 0; i < n; i++){
        cin >> v[i];
    }
    sort(v.begin(), v.end());
    ull sum = 0;
    for (int i = 0; i < n; i++){
        sum += 1LL * (2 * i - n + 1) * v[i];
    }
    cout << sum;
    return 0;
}
```

6. H001 – Tom’s Currency

➤ Đề bài:

University of Information Technology, VNU-HCM
UIT Algo Bootcamp

Rookie Training
Spring 2023

Tom và tiền tiết kiệm

Sau một thời gian dài tích góp, Tom đang có trong tay n tờ tiền, tờ tiền thứ i mang mệnh giá v_i . Một ngày đẹp trời, Tom quyết định đem hết số tiền bản thân tích góp được đi gửi ngân hàng với ước mơ trở thành tỷ phú.

Tình cờ thay, hôm đó là ngày đầu tiên Jerry đến nhận việc tại ngân hàng và Tom chính là vị khách hàng đầu tiên mà Jerry phục vụ. Để trách việc nhầm lẫn không đáng có, việc đầu tiên Jerry cần làm là tính số lượng mệnh giá tiền phân biệt mà Tom gửi vào ngân hàng.

Do Jerry là nhân viên mới, còn thiếu kinh nghiệm nên mọi người hãy giúp Jerry nhé!

Input

- Dòng đầu tiên chứa số n là số lượng tờ tiền mà Tom có. ($1 \leq n \leq 3 \times 10^5$)
- Dòng thứ hai chứa n số nguyên dương, lần lượt là $v_1, v_2, \dots, v_n$. ($1 \leq v_i \leq 10^9$)

Output

- Dòng đầu ghi số nguyên dương k là số mệnh giá tiền phân biệt mà Tom gửi vào.
- Dòng thứ hai ghi k số nguyên dương $b_1, b_2, \dots, b_k$ là các mệnh giá tiền được sắp xếp theo thứ tự tăng dần.

Example

input	output
6	4
3 100 2 3 3 10	2 3 10 100

➤ Ý tưởng:

Yêu cầu đưa ra độ dài và mệnh giá của những tờ khác nhau nên ta nghĩ ngay tới cấu trúc dữ liệu **set**.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    set<int> s;

    for(int i = 0; i < n; i++) {
        int temp;
        cin >> temp;
        s.insert(temp);
    }

    cout << s.size() << '\n';
    for (int i : s)
        cout << i << " ";

    return 0;
}
```

7. Hashing: VQ44_FLOWERS

➤ Đề bài:

VQ44. ĐÈN HOA

Tên chương trình: FLOWERS.???

Để chuẩn bị một ngày lễ lớn lần thứ k thành phố quyết định dùng đèn LED kết thành k bông hoa trang trí dọc đường phố chính, mỗi bông hoa được kết từ một loại bóng LED cùng màu. Để có hiệu ứng ánh sáng tốt nhất Công ty Chiếu sáng được yêu cầu tạo ra được càng nhiều hoa khác màu càng tốt. Trong kho của Công ty có n bộ đèn LED mỗi bộ lắp được một đèn và bộ thứ i có màu a_i .

Hãy chỉ ra k màu của các bộ đèn cần chọn sao cho số lượng hoa khác màu lắp được là nhiều nhất. Nếu có nhiều cách khác nhau thì đưa ra cách tùy chọn.

Dữ liệu: Vào từ file văn bản FLOWERS.INP:

- ✚ Dòng đầu tiên chứa 2 số nguyên n và k ($1 \leq k \leq n \leq 10^5$),
- ✚ Dòng thứ 2 chứa n số nguyên $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9, i = 1 \div n$).

Kết quả: Đưa ra file văn bản FLOWERS.OUT trên một dòng k số nguyên – màu của các đèn được chọn.

Ví dụ:

FLOWERS.INP									
10	4								
8	8	8	8	8	8	8	8	2	1

FLOWERS.OUT			
1	2	8	8



Vq44_OI20150315_A

➤ Ý tưởng:

Đề bài yêu cầu ta dùng Hash dạng map, vậy thì ta sẽ đếm số lượng của các đèn mỗi màu, sau đó chọn ra mỗi màu 1 đèn để đưa vào vector kết quả, lặp lại tới khi hết số lượng màu khác nhau thì ta sẽ thêm các đèn còn lại vào đến khi đủ.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int n, k;
vector<int> a;

vector<int> get_ans(const vector<int>& A,int K){
    map<int,int> Hash;
    vector<int> res;
    for(int i : A) {
        Hash[i]++;
    }
    while(res.size() < K) {
        for(auto& x : Hash) {
            if(res.size() < K && Hash[x.first] > 0) {
                res.push_back(x.first);
                Hash[x.first]--;
            }
            else if (res.size() >= K) break;
        }
    }
    return res;
}

int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);

    cin >> n >> k;

    a.resize(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    vector<int> ans = get_ans(a,k);

    for (const int& x: ans) cout << x << ' ';
    return 0;
}
```

8. Hashing: KiemKe

➤ Đề bài:

Kiểm kê

Cửa hàng X có nhiều loại hàng, mỗi loại hàng đều có mã khác nhau. Tuy nhiên, trong lúc bán hàng, nhân viên quên không nhập số lượng mỗi loại mà chỉ nhập mã các loại hàng trên từng dòng. Bạn hãy đếm xem có bao nhiêu loại hàng khác nhau?

Ví dụ: Có một số loại hàng có mã lần lượt là [123, 45, 8, 19, 1, 2333].

Bạn nhân viên sẽ nhập theo từng dòng [123, 45, 45, 8, 1, 123, 123, 8, 8].

Số loại hàng khác nhau trong dãy được nhập sẽ là 4.

Input:

- Dòng đầu tiên là số nguyên N ($1 \leq N \leq 5 \times 10^4$) là số lượng mã loại hàng được nhập.
- N dòng tiếp theo, mỗi dòng là một mã của loại hàng được nhập, độ dài của mã có thể lên tới 100.

Output:

- Một dòng chứa số nguyên là số các loại hàng khác nhau.

➤ Ý tưởng:

Vì bài yêu cầu dùng hashmap nên ta cũng sẽ duyệt qua số mã nhập vào và thêm số lượng cho từng mã khác nhau, đến cuối trả về kích thước hashmap là được.

➤ Script:

```
#include <bits/stdc++.h>
using namespace std;

int N;

int count_distinct(const vector<string>& ids){

    map<string,int> Hash;
    int count = 0;

    for(string s : ids)
        Hash[s]++;

    for(auto& x : Hash)
        count++;
    return count;
}

int main(){
    ios::sync_with_stdio(false); cin.tie(nullptr);
    cin >> N;
    vector<string> ids(N);

    for (int i = 0; i < N; i++) cin >> ids[i];

    cout << count_distinct(ids);

    return 0;
}
```